

Project Report: Student Result Management System

Track Name: Python GUI

Developer:

1. Jannat Nur Urme (22-47186-1)
2. Sanjida Chowdhury(23-52842-2)
3. Nabila Hoque (23-50895-1)
4. Disha, Amrita Bhawal(23-50892-1)

1. Project Overview

The Student Result Management System is a desktop-based application developed using Python and the Tkinter GUI library. The application allows teachers or administrators to manage student academic records efficiently through an easy-to-use graphical interface. Users can add, search, update, delete, and view student records while the system automatically calculates total marks, percentage, grades, and pass/fail status.

The application stores all records in a JSON file, making the data permanent even after the program is closed. It also generates class statistics such as average percentage, highest and lowest marks, pass rate, failed students, and standard deviation using the NumPy library.

2. System Architecture & Tech Stack

The Student Result Management System is a standalone desktop application designed to automate student result processing.

Frontend GUI

Built using Python's **Tkinter** library to provide a simple and user-friendly graphical interface. The **ttk.Treeview** widget is used to display student records in a tabular format.

Backend

Python handles the business logic, student management, calculations, and file operations.

Database

The application uses a lightweight **JSON** file to permanently store student records.

Design Pattern

The system follows a modular design by separating the GUI, business logic, calculations, student model, and file handling into different Python modules.

Libraries Used

- tkinter
- tkinter.ttk
- tkinter.messagebox
- tkinter.simpledialog
- json
- NumPy

3. Detailed Module Breakdown

Module 1: Main Program

This module is the entry point of the application.

It creates the Tkinter root window, initializes the ResultManagerApp, and starts the application's main event loop.

Responsibilities

- Start the application
- Create the main window
- Load the GUI
- Connect frontend with backend modules

Student Result Manager & Analyzer											
Manage student marks · Calculate grades · Analyse class performance											
MENU											
Add Student											
View All											
Search											
Update											
Delete											
Statistics											
Save Data											
Load Data											
Exit											
Tip: Double-click a row to view details. Select a row then Update/Delete it.											
Quick Filter: Clear											
12 record(s)											
ID	Name	Maths	Eng	Phy	Chem	CS	Total	Percentage	Grade	Status	
S001	mina	85	78	90	88	92	433	86.60%	A	Pass	
S002	rina	45	60	38	42	55	240	48.00%	F	Fail	
S003	meera	72	68	75	70	80	365	73.00%	B	Pass	
S004	hafsaa	95	92	88	96	98	469	93.80%	A+	Pass	
S005	shojol	55	58	62	50	65	290	58.00%	D	Pass	
S006	fahmida	30	25	40	35	20	150	30.00%	F	Fail	
S007	shawon	80	85	78	82	88	413	82.60%	A	Pass	
S008	ontora	62	70	58	65	72	327	65.40%	C	Pass	
S009	adiba	48	52	45	50	55	250	50.00%	D	Pass	
S010	labib	91	88	85	90	87	441	88.20%	A	Pass	
S011	labiba	25	30	20	35	28	138	27.60%	F	Fail	
S012	prona	78	82	76	80	84	400	80.00%	A	Pass	

Module 2: User Interface

This module provides the complete graphical user interface of the application. It allows users to interact with the system using buttons, forms, tables, and dialog boxes.

Features

- Add Student
- View All Students
- Search Student
- Update Student
- Delete Student
- Save Data
- Load Data
- Class Statistics
- Quick Filter
- Input Validation

Add Student

Users enter:

- Student ID
- Student Name
- Mathematics
- English
- Physics
- Chemistry
- Computer Science

The system validates the inputs before adding the record.

The screenshot displays the 'Student Result Manager & Analyzer' application. A modal window titled 'Add New Student' is open, allowing users to input student information. The form includes fields for 'Student ID', 'Full Name', and 'Subject Marks (0 - 100)' for Mathematics, English, Physics, Chemistry, and Computer Science. A green 'Add Student' button is at the bottom of the modal. In the background, a table shows student marks for various subjects. A tip at the bottom left states: 'Tip: Double-click a row to view details. Select a row then Update/Delete it.'

	Maths	Eng	Phy	Chem
	85	78	90	88
	45	60	38	42
	72	68	75	70
	95	92	88	96
	55	58	62	50
	30	25	40	35
	80	85	78	82
	62	70	58	65
	48	52	45	50
	91	88	85	90
	25	30	20	35
	78	82	76	80

Search Student

Allows users to search a student by Student ID or part of the student's name.

Search Student

×

Enter Student ID or part of name:

OK

Cancel

Update Student

Allows users to update an existing student's information using the Student ID.

Student Result Manager & Analyzer

Manage student marks · Calculate grades · Analyse class performance

MENU

Add Student

View All

Search

Update

Delete

Statistics

Save Data

Load Data

Exit

Tip: Double-click a row to view details. Select a row then Update/Delete it.

Quick Filter: Clear

ID	Name	Maths	Eng	Phy	Chem	CS
S001	mina	85	78	90	88	92
S002	rina	45	60	38	42	55
S003	meera	72	68	75	70	80
S004	hafsa	95	92	88	96	98
S005	shojol	55	58	62	50	65
S006	fahmida	30	25	40	35	20
S007	shawon	80	85	78	82	88
S008	ontora	62	70	58	65	72
S009	adiba	48	52	45	50	55
S010	labib	91	88		90	87
S011	labiba	25	30		35	28
S012	proma	78	82		80	84

Update Student

×

Enter the Student ID to update:

OK

Cancel

Module 3: Student Record Management

This module contains the StudentManager class responsible for managing all student records.

Features

- Add Student
- View Student
- Search Student
- Update Student
- Delete Student
- Duplicate Student ID Check
- Save Student List
- Load Student List

The manager module acts as the bridge between the GUI and the JSON database.

Student Result Manager & Analyzer											
Manage student marks · Calculate grades · Analyse class performance											
Quick Filter: Clear											
12 record(s)											
ID	Name	Maths	Eng	Phy	Chem	CS	Total	Percentage	Grade	Status	
S001	mina	85	78	90	88	92	433	86.60%	A	Pass	
S002	rina	45	60	38	42	55	240	48.00%	F	Fail	
S003	meera	72	68	75	70	80	365	73.00%	B	Pass	
S004	halsa	95	92	88	96	98	469	93.80%	A+	Pass	
S005	shojol	55	58	62	50	65	290	58.00%	D	Pass	
S006	fahmida	30	25	40	35	20	150	30.00%	F	Fail	
S007	shawon	80	85	78	82	88	413	82.60%	A	Pass	
S008	ontora	62	70	58	65	72	327	65.40%	C	Pass	
S009	adiba	48	52	45	50	55	250	50.00%	D	Pass	
S010	labib	91	88	85	90	87	441	88.20%	A	Pass	
S011	tabiba	25	30	20	35	28	138	27.60%	F	Fail	
S012	prona	78	82	76	80	84	400	80.00%	A	Pass	

Module 4: Student Class & File Handling

This module implements the Object-Oriented Programming structure of the application.

Student Class

The Student class stores all student-related information including:

- Student ID
- Student Name
- Subject Marks
- Total Marks
- Percentage
- Grade
- Pass/Fail Status

File Handling

The file_handler.py module manages permanent data storage.

Functions include:

- Save data into JSON
- Load data from JSON

- Read JSON files
- Write JSON files
- Handle file-related exceptions

All records are stored inside students_data.json.

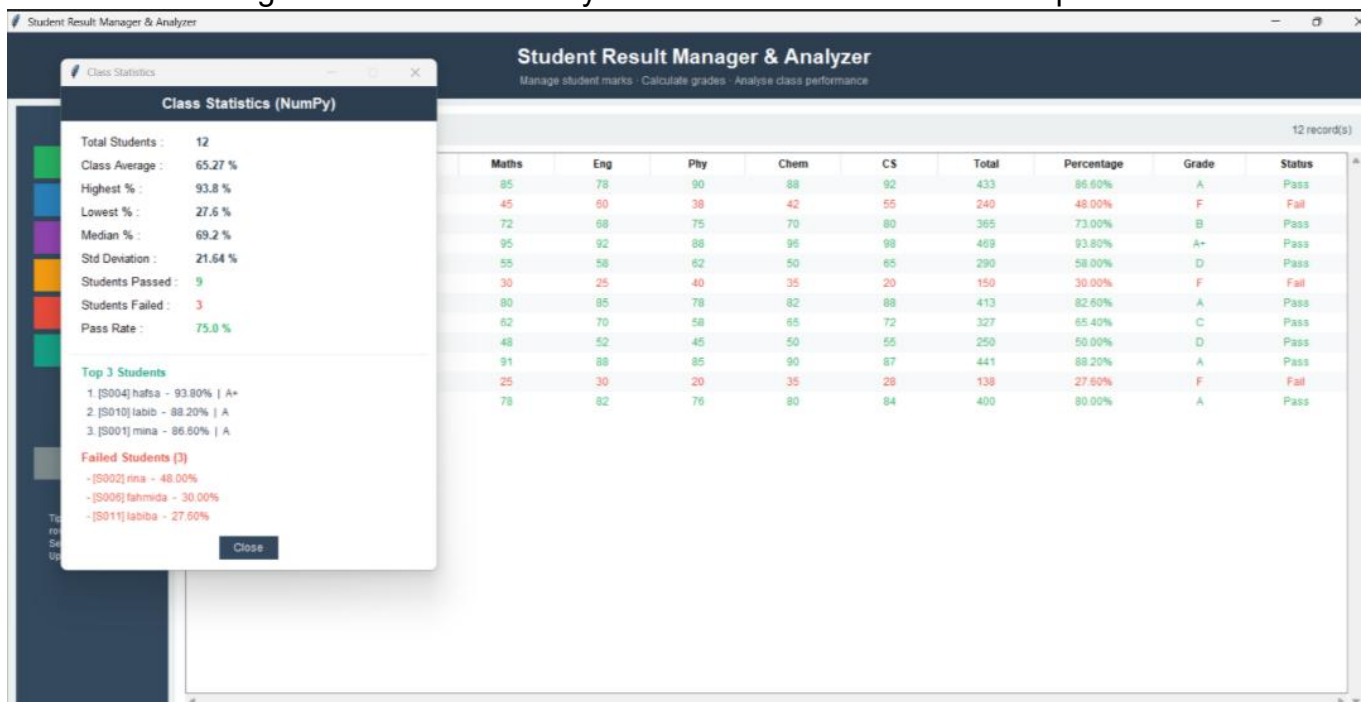
Module 5: Result Analysis

This module performs all academic calculations.

Features

- Total Marks
- Percentage Calculation
- Grade Calculation
- Pass/Fail Status
- Average Percentage (NumPy)
- Highest Percentage
- Lowest Percentage
- Median Percentage
- Standard Deviation
- Top 3 Students
- Failed Student List
- Pass Rate

The statistics are generated automatically whenever student records are updated.



Module 6: JSON Database

The project uses a JSON file as its database.

The database stores:

- Student ID
- Student Name

- Subject Marks
- Total Marks
- Percentage
- Grade
- Pass/Fail Status

4. Object-Oriented Programming (OOP)

The Student Result Management System is developed using a modular Object-Oriented Programming approach.

A. Class-Based Design (Encapsulation)

The Student class encapsulates all information related to an individual student, while the StudentManager class encapsulates all operations such as adding, updating, searching, and deleting student records.

B. Modular Design

The project is divided into independent modules:

- GUI Module
- Student Model
- Student Manager
- Result Calculation
- File Handling

This modular design improves readability, maintenance, and scalability.

C. Abstraction

The GUI interacts only with the StudentManager class. The user interface does not directly access the JSON file or perform calculations, as these responsibilities are handled by dedicated backend modules.

D. Reusability

Calculation functions such as total marks, percentage, grade, and statistics are implemented in a separate module (calculations.py), allowing them to be reused throughout the application.

5. System Workflow

User Opens Application



Main Dashboard Opens



User Selects an Operation



Enter or Modify Student Information



Input Validation



StudentManager Processes Data



Calculations Performed

↓
Data Saved to JSON
↓
Student Table Updated
↓
Statistics Generated

6. Advantages

- Simple and user-friendly graphical interface.
 - Automatic calculation of total marks, percentage, grades, and pass/fail status.
 - Permanent data storage using JSON.
 - Duplicate Student ID prevention.
 - Quick filtering and searching of student records.
 - Automatic class statistics generation.
 - Modular and maintainable code structure.
-

7. Limitations

- Offline desktop application.
 - Supports only a single user at a time.
 - Uses JSON instead of a relational database.
 - No login or authentication system.
 - No report printing or PDF export.
-

8. Future Improvements

The project can be enhanced by adding:

- SQLite or MySQL database support.
 - User login and authentication.
 - PDF result report generation.
 - Excel import/export.
 - Graphical performance charts.
 - Cloud storage integration.
 - Multi-user support.
 - Backup and restore functionality.
-

9. Conclusion

The Student Result Management System successfully demonstrates a Python Tkinter-based desktop application for managing student academic records. It automates result calculation, simplifies record management, and provides useful statistical analysis through a clean graphical interface. By applying Object-Oriented Programming principles and a modular design, the system is easy to maintain, scalable, and serves as a strong foundation for future academic management applications.